

Making-of: How Stigix Powered the Prisma SD-WAN Failover Tests

Sep 25, 2026 · Jean-Louis SUZANNE, Technical Sales Manager, Palo Alto Networks

Why a test platform

The white paper *When Routing Can't See the Failure* shows that Prisma SD-WAN fails over when a data center's internet breakout dies, even though every route and tunnel stays up. This making-of explains how those results were produced, and what [Stigix](#) did at each step.

Failover testing is easy to do badly. Pull a cable, refresh a web page, and you learn almost nothing: not when the failure started, not which path the traffic took, not what users actually felt. A useful test needs four things:

- **Realistic traffic** running before, during and after the failure, so there is something to fail over.
- **Controlled, repeatable failures**, injected at a known point and a known time.
- **Measurements from several angles**: packets, application sessions, voice quality, and the SD-WAN's own path decisions.
- **A timeline** that ties all of these together to the second.

Stigix provided all four from one place. And because it exposes its functions through the Model Context Protocol (MCP), the whole campaign could be driven by an AI assistant, Claude, under an engineer's supervision.

The lab as Stigix sees it

Stigix runs as a small agent on a Linux host at each site. The agents form a mesh, with DC1's node acting as leader, and each can generate traffic toward the others. On top of the SD-WAN itself, three Stigix integrations did the heavy lifting.

Stigix nodes (traffic endpoints):

Node	Address	Role in the tests
BR8-Ubuntu	192.168.219.1	Test source: all traffic, probes and measurements start here

Node	Address	Role in the tests
DC1-Ubuntu	192.168.203.100	Target in DC1 for convergence probes, speed tests and TCP apps; mesh leader
BR1, BR2, BR5 nodes	192.168.207.10, 192.168.206.10, 192.168.217.5	Control targets: voice calls and TCP sessions that should not be affected

VyOS routers (failure injection). The lab's WAN and data center networks are emulated with VyOS routers that Stigix can drive:

Router	What it emulates	Interfaces used in the tests
vyosrouter	The underlay: every site's internet and MPLS circuits	eth10, DC1-INET-221 (DC1's internet circuit, carrying the SD-WAN tunnels)
vyoslandc1	DC1's LAN and central internet breakout	eth3, DC1 INTERNET EXIT (the breakout behind DC1)

The distinction matters. Cutting eth10 on vyosrouter kills the *tunnels* to DC1, which the SD-WAN sees within seconds. Cutting eth3 on vyoslandc1 kills only the *breakout behind* DC1, which no tunnel or route reveals. The white paper is about the second case.

Prisma SD-WAN integration. Stigix queries the Prisma SD-WAN Flow Browser through the Prisma SASE API. For every flow, it returns the chosen path, the preferred path, the path decision history, and packet and byte counters. This turned the SD-WAN's own decisions into test evidence.

The Stigix toolbox used in the tests

Each Stigix capability answered a different question. Together, they gave a complete picture of every failure.

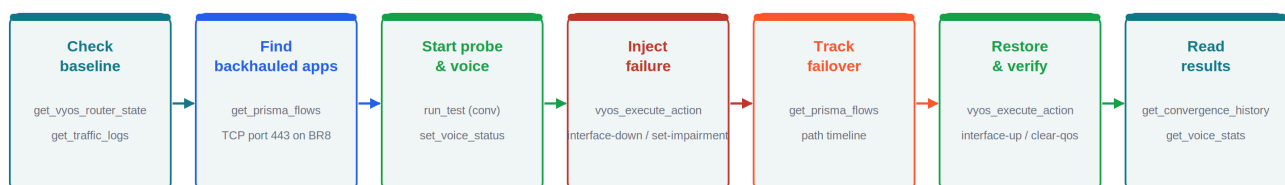
Stigix capability	What it did	What it proved
Application traffic generator	Continuous simulated SaaS traffic from BR8 (Microsoft 365, Google, Salesforce, Dropbox and more), each request logged with its HTTP code and timing	Backhauled apps failing (timeouts) during the outage, and recovering after failover

Stigix capability	What it did	What it proved
Convergence probes	A UDP flow at 50 packets per second from BR8 to DC1, echoed back	Maximum blackout, loss per direction (in % and ms), verdict
Voice simulation	30-second G.711 calls from BR8 to DC1, BR1 and BR2	Loss, jitter and MOS per call, e.g. MOS 1.48 for a call caught by the outage
Custom TCP applications	Persistent TCP sessions from BR8 to DC1, BR2 and BR5	Sessions to DC1 break on failover but survive failback; control sessions untouched
Speed tests	TCP and UDP throughput tests, upstream and downstream	Link capacity baseline, up to about 200 Mbit/s upstream
VyOS control	Interface shutdown and restore, latency and loss injection, cleanup, with exact commit timestamps	Failure start and end times, to the millisecond
Prisma SD-WAN Flow Browser queries	Flows filtered by address, port and protocol, with each flow's path decisions	Which path the SD-WAN chose, and exactly when it changed
Test history and UI	The Failover Lab view, with each test's failover path sequence	Shareable evidence, including exportable PoC cards

The last two rows are what make Stigix more than a traffic generator. The traffic shows *what users felt*; the Flow Browser data shows *what the SD-WAN decided*. Lining them up on one timeline is what turns an observation into an explanation.

Step by step: the internet breakout test

Test sequence: Stigix MCP calls step by step



This is how the test in the white paper actually ran, with the Stigix function behind each step.

Step	Stigix function	Purpose
1. Check application health	<code>get_traffic_logs</code> on BR8	Confirm every simulated app answers before the test
2. Find the backhauled apps	<code>get_prisma_flows</code> on site BR8, TCP port 443	See which apps go through the tunnel to DC1 and which go direct
3. Record the router baseline	<code>get_vyos_router_state</code> on vyoslandc1	Know exactly what to restore afterwards
4. Phase A: hard failure	<code>vyos_execute_action</code> <code>interface-down</code> on eth3	Kill DC1's breakout
5. Phase B: silent failure	<code>vyos_execute_action</code> <code>set-impairment</code> , 100% loss on eth3	Kill the breakout while the interface stays up
6. Follow the failover	<code>get_prisma_flows</code> filtered on one app's destination, with the aggregated path timeline	Date every path change
7. Confirm the user impact	<code>get_traffic_logs</code> on BR8	See apps fail, then recover
8. Verify the restore	<code>get_vyos_router_state</code> on vyoslandc1	Compare with the baseline

Finding what to watch

Step 2 was the key preparation. One query on BR8's HTTPS flows showed that four applications went through BR8-INET2 → DC1-INET: Exchange Online, Microsoft Entra ID, Microsoft Teams and a custom app hosted on Cloudflare. Everything else left BR8 directly. Without this, we would not have known which traffic the failure would hit.

Reading a failure in the Flow Browser

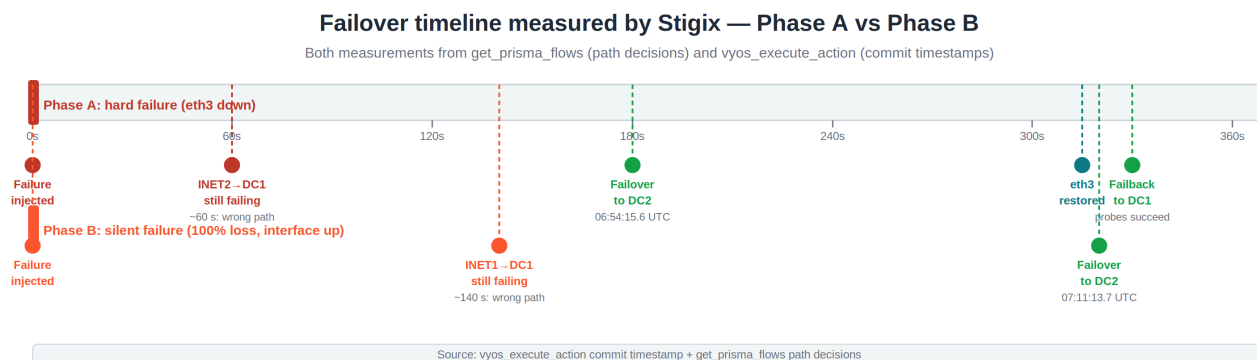
During the outage, a failing session is easy to spot in Stigix's flow data: 2 packets sent, 120 bytes, 0 bytes back. Those are the SYN and its retransmission, never answered. When the path changes, the same query shows new sessions with full responses on the DC2 tunnels. That contrast gave the exact failover time, to the millisecond: **06:54:15.6 UTC** in Phase A, **07:11:13.7 UTC** in Phase B.

What the injection looks like on the router

For Phase B, Stigix translated `set-impairment` into this VyOS configuration on `vyoslandc1`:

```
set qos policy network-emulator LAB_COMBINED_eth3 loss '100'
set qos interface eth3 egress 'LAB_COMBINED_eth3'
```

The interface stays up and the routing table does not change. Only the packets disappear, which is exactly the grey failure we wanted.



The earlier campaign: when the tunnel itself fails

Before the breakout test, we ran eight failovers of a different kind: cutting DC1's internet circuit (`eth10` on `vyosrouter`), which takes the SD-WAN tunnels down with it. This campaign set the reference point the white paper compares against.

Each run followed the same Stigix recipe: start voice calls, start a convergence probe from BR8 to DC1, cut `eth10`, restore it, stop the probe, then read the results.

Measurement	Stigix source	Result
Maximum blackout	Convergence probe	7.7 to 9.3 s over eight runs, 8.6 s on average
First path decision after the cut	VyOS commit timestamp + Flow Browser	0.9 to 1.8 s (two runs with exact timestamps)
Loss per direction	Convergence probe (echo)	~3 s upstream, ~5 s downstream
+200 ms latency, no cut	VyOS impairment + Flow Browser	No path change, in every run where it was applied
Voice call caught by the cut	Voice simulation	26% loss, MOS 1.48

Measurement	Stigix source	Result
Voice call during the outage, via DC2	Voice simulation	No loss, MOS 4.4
TCP sessions to DC1	Custom TCP apps	Dropped at the cut, reconnected in ~3 s via DC2; kept on failback
TCP sessions to BR2 and BR5	Custom TCP apps	Untouched

Two Stigix features made this campaign much more precise than a manual test:

- **The echo in the convergence probe** separates upstream from downstream loss. That is how we found that replies from DC1 take about 2 seconds longer to recover than traffic toward it.
- **Exact VyOS commit timestamps** separate the SD-WAN's *decision* (under 2 s) from the *actual recovery* of traffic (about 8 s). Without them, both blur into one number.

The voice and TCP results also changed the story told to customers: the real impact of a failover is not the path change itself, but the reconnection of the applications that were running at that moment.

Measurement tricks and pitfalls

Most of the rigor in these tests came from lessons learned the hard way. They apply to anyone testing SD-WAN failover with Stigix, or with any other tool.

Find the test flow by its source port. Each Stigix convergence probe uses a fixed source port derived from its test number: CONV-0250 uses port 30250. Stigix exposes it in the test history, so you can ask the Flow Browser for that exact flow instead of searching through hundreds.

The Flow Browser runs about one minute behind. A query made during the outage often shows the flow as it was before the failure. Keep polling, or read again after the test.

On a long-lived flow, the path history can be overwritten. When the link came back while a probe was still running, the failback decision replaced the earlier failover in the flow's record. The failover then vanished from the history, in Stigix and in Prisma SD-WAN's own interface. Two workarounds: stop the probe before restoring the link, or poll repeatedly during the test and keep every decision you see. We reported this behavior to Palo Alto Networks.

Check the path policy first. At the start, the test flow showed a preferred path of “N/A”, and failovers were not recorded on it. The policy referenced paths that BR8 does not have. Once it listed only BR8’s real paths, the failover sequence appeared as expected.

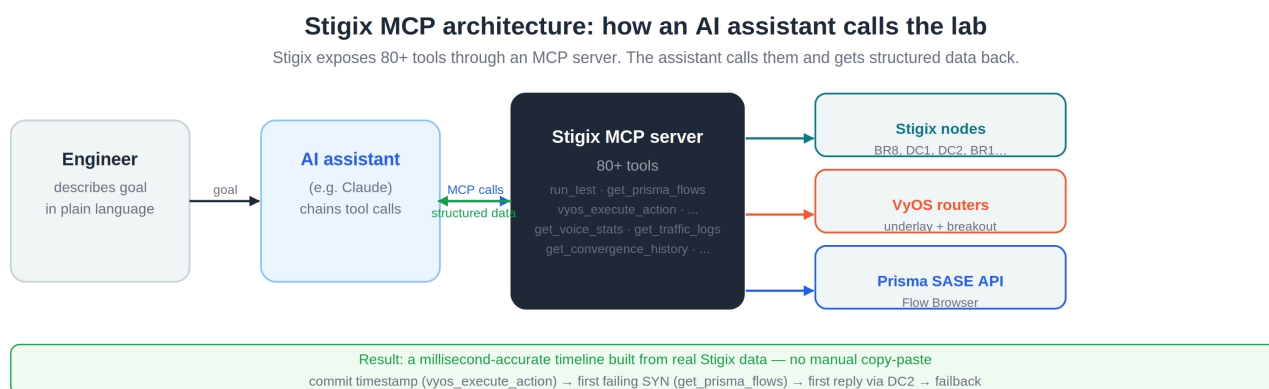
Beware the traffic generator’s backoff. When an application fails, the generator pauses it for 5 minutes to avoid flooding. During an outage, the failing apps therefore disappear from the flow data. We used the Cloudflare-hosted app, which kept a steady rate of new sessions, as the reference for all timings.

Single-packet flows are noise, and a free sampler. Stigix’s dashboard sends reachability probes to port 6200 every few seconds. They clutter queries on that port, so Stigix filters them out by default. But since each probe is a new flow with its own path decision, they also show which path new traffic would take at any moment.

An HTTP error is not a network error. Microsoft 365 answered the generator with codes like 401 or 417 because requests are not authenticated. That still proves the network path works. A real failure shows as code 000: no answer at all.

Never trust a single speed test. On the same path, BR8-to-DC1 throughput swung between about 10 and 200 Mbit/s from one run to the next. UDP tests also hit a ceiling of about 85 Mbit/s set by the sending host, not by the network. Repeat, and compare directions.

Stigix's MCP interface



Beyond the web interface, Stigix exposes its full capability set through a [Model Context Protocol](#) (MCP) server. MCP is an open standard that lets AI assistants connect to external tools and call them in context. Stigix implements more than 80 MCP tools, covering every function used in these tests.

The MCP interface makes Stigix **orchestratable in natural language**. An engineer describes a scenario — “start a convergence probe from BR8 to DC1, then inject 100% loss on DC1’s internet exit and track the failover” — and an AI assistant, such as Claude, chains

the exact Stigix calls, reads the results, and builds the timeline. The assistant has no knowledge of the network beyond what Stigix returns: every decision, every timestamp and every path comes from a real Stigix query on the live lab.

What the MCP tools covered in these tests:

MCP tool	What it does
<code>run_test</code>	Start a convergence probe, speed test or voice call
<code>stop_test</code>	Stop the probe and collect final metrics
<code>get_test_status</code>	Read live metrics during the test
<code>get_convergence_history</code>	Read the failover path sequence after the test
<code>get_voice_stats</code>	MOS, loss and jitter per simulated call
<code>get_traffic_logs</code>	Application-level results from the traffic generator
<code>get_prisma_flows</code>	Query the SD-WAN Flow Browser, filter by address, port or source port
<code>get_vyos_router_state</code>	Record the router's baseline before and after each test
<code>vyos_execute_action</code>	Inject failures: <code>interface-down</code> , <code>set-impairment</code> , <code>clear-qos</code>
<code>list_custom_tcp_apps</code> , <code>get_tcp_app_sessions</code>	Track persistent TCP sessions through the failover
<code>set_voice_status</code>	Start or stop the voice simulation on a node
<code>get_health_matrix</code>	Check node CPU, bitrate and overall health

The key value of the MCP layer is **traceability**. Every call returns structured data, so the assistant can cross-reference the router's commit timestamp (from `vyos_execute_action`) against the first failing session (from `get_prisma_flows`) and produce a timeline accurate to the millisecond, without any manual copy-pasting.

The MCP server also enforces its own guardrails. Disruptive tools such as `interface-down` include explicit warnings that prompt the engineer for confirmation before the call proceeds. If any tool call fails, the assistant stops the sequence and reports the state of the lab.

Reproduce it yourself

Stigix is available at github.com/jsuzanne/stigix. Here is a checklist to run the same tests on your own lab.

Lab prerequisites

- ☐ A Prisma SD-WAN branch and two data centers, with the backup DC in the application path policy.
- ☐ A Stigix node at the branch and in each data center.
- ☐ VyOS routers emulating the underlay and the data center breakout, registered in Stigix.
- ☐ Prisma SASE API access configured in Stigix, for Flow Browser queries.

Before the test

- ☐ Check that the path policy only lists paths the branch really has (no “N/A” preferred path on test flows).
- ☐ Make sure router management does not depend on the interface you will cut.
- ☐ Identify the backhailed applications from the branch's flows, and pick one with steady traffic as your reference.
- ☐ Record the routers' state as a baseline.

During the test

- ☐ Start voice calls and a convergence probe, then inject the failure.
- ☐ Poll the Flow Browser every 30 to 60 seconds, and keep every path decision you see.
- ☐ To keep the failover in the flow's history, stop the probe before restoring the link.
- ☐ Restore, then wait for the probes to bring traffic back.

After the test

- ☐ Compare the routers with the baseline, including empty QoS containers left by cleanup.
- ☐ Read the test history about a minute after stopping, once the path sequence is filled in.
- ☐ Export the PoC card from the Stigix interface to share the results.